

Penney's Game 与 Conway 公式

第二部分 —— 一份完整的推导与翻车历程

从联合马尔可夫链的可靠解，到一条自创递归的彻底翻车，
再到矩阵闭式与鞅的正解；以及多个 AI 互相拆台 + 代码裁判

如何把一个被记忆带偏的结论重新救回来

读这份讲义之前，先知道它是什么

这不是一份“标准答案”式的讲义。它是一份**真实的探索手稿**：记录我们如何从第一性原理一步步推 Penney's game 的赔率公式，如何**推岔进一条死路**、花了七八个小时、最后被一行代码证伪，又如何三个 AI (Claude、GPT、DeepSeek) 互相拆台、加上 Python 联合链做“裁判”之下，定位错误、回到正解。

我们**故意保留了所有弯路**——包括 Claude 凭记忆给出一个错误公式还信誓旦旦说“很明显”、被一个两字符的反例当场打脸的全过程。因为对读者而言，“**为什么这样推会错**”比“**正确答案是什么**”珍贵得多。课本只给你擦干净的结论；这份讲义给你结论诞生前的所有泥巴。

阅读约定。全文有四种颜色的盒子：

- **紫色·直觉**：在写公式之前，先用大白话/类比讲“为什么应该是这样”。
- **灰色·尝试**：我们当时真的写下的某一步（可能对、可能错）。
- **橙色·翻车**：这一步错在哪、被什么打脸。
- **绿色·想通**：纠正之后真正理解了什么。

Contents

1 热身：游戏规则与两个反直觉的问题	2
1.1 游戏是什么	2
1.2 两个让人栽跟头的问题	3
2 可靠解：联合马尔可夫链	3
2.1 为什么状态要“两个进度一起记”	3
2.2 一个失配要“退回去”：KMP 的回退	4
2.3 动手算：HHH 对 THH	4
2.3.1 第一步：列出所有“还没分胜负”的状态	4
2.3.2 第二步：写下每个状态读 H / 读 T 去哪	5
2.3.3 第三步：把“胜率”写成方程组	5
2.3.4 第四步：解方程	5
3 翻车历程：试图找一个“闭式公式”的那条死路	6
3.1 弯路一： $P(\text{长度} = n) = \frac{1}{2^n}$ 吗？	6
3.2 弯路二：生存量漏掉了对手 Y	7
3.3 弯路三： (k, p) 划分不互斥（“相容”陷阱）	7
3.4 弯路四：一条“看起来很漂亮”的自创递归	8
3.5 弯路五：Claude 凭记忆给出一个假公式，还说“很明显”	9

4 验证危机：代码裁判一锤定音	9
4.1 用”裁判”检验：不等长的例子	9
4.1.1 裁判给出真值	10
4.1.2 我们的递归给出什么	10
5 正解：局面、矩阵闭式与鞅	10
5.1 第一处错：只记 $\Gamma_m = j$ 不够，要记完整局面	10
5.2 第二处错：角色轮换把”进度”误当成”新模式”	11
5.3 看一张图：HH 对 HTH 的联合链	11
5.4 矩阵闭式：一个公式搞定等长与不等长	12
5.4.1 安全转移矩阵 M	12
5.4.2 前段计数： $N_m = e_0 M^m$	12
5.4.3 最后冲刺筛选 R_X ：替换错误的 χ_j	12
5.4.4 合起来：闭式	13
5.5 鞅到底在哪里	13
5.5.1 三个极易混淆的对象	13
5.5.2 F 满足 Bellman 方程，且 $F(S_{t \wedge \tau})$ 是严格鞅	13
5.5.3 odds 是两个鞅初值之比	14
6 三种情形与那个漂亮的 Conway 公式	14
6.1 情形一：两模式完全相同	14
6.2 情形二：等长但不同——Conway 漂亮公式	15
6.3 情形三：不等长——不能硬套 Conway 差值	15
6.4 顺便：那个被丢弃的 K 和真 Conway C 的关系	16
7 回到问题二：没有最强模式（非传递性）	16
8 元教训：当来源都会犯错，如何逼近真相	16
8.1 完整的”翻车—救回”链条	16
8.2 四条可迁移的纪律	17
8.3 真正沉淀下来的数学	17
8.4 留待第三部分的方向	18

第1章 热身：游戏规则与两个反直觉的问题

1.1 游戏是什么

抛一枚公平硬币，正面 H、反面 T，每次各 $\frac{1}{2}$ ，一直抛下去，得到一条无限长的序列，比如

HTTHTHHT...

现在有两个玩家，各自事先选定一个**固定的模式**（pattern），也就是一小段 H/T 的串。比如玩家甲选 HHH，玩家乙选 THH。然后大家盯着这条不断变长的序列：**谁选的模式先作为一段连续子串出现，谁就赢。**

用大白话说

想象两个人在听同一段莫尔斯电码。甲在等”滴滴滴”，乙在等”嗒滴滴”。电码一个符号一个符号地来，第一个等到自己那串的人赢。注意两人听的是**同一条流**——这是整道题所有微妙之处的来源：你往前凑你的模式时，吐出来的符号同时也在帮（或不帮）对手凑他的模式。

我们把”模式 X 第一次出现的结束位置”记作一个随机变量

$$T_X = \min\{t : \omega_{t-|X|+1}\omega_{t-|X|+2}\cdots\omega_t = X\},$$

读作： T_X 是最小的那个时刻 t ，使得序列在第 t 位结尾的那一小段恰好拼成 X 。（ $|X|$ 表示 X 的长度。）于是” X 先于 Y 出现”就是事件 $\{T_X < T_Y\}$ ，我们要算的核心量是

$$V(X, Y) = \mathbb{P}(T_X < T_Y) \quad (”X 先赢”的概率)。$$

1.2 两个让人栽跟头的问题

问题一（怎么算）：给定两个模式 X, Y ，怎么算出 $V(X, Y)$ ？它和”平均要等多久才第一次看到 X ”（也就是期望 $\mathbb{E}[T_X]$ ）是不是一回事？

问题二（有没有最强）：是不是存在一个”天下无敌”的模式，对谁都能赢？换句话说，”挑那个平均等待时间最短的模式”是不是必胜策略？

第一部分给我们埋下的两颗种子

在第一部分（等待 HHH / HTH）里我们算出：平均等到 HHH 要 $\mathbb{E}[T_{\text{HHH}}] = 14$ 步，而平均等到 HTH 只要 $\mathbb{E}[T_{\text{HTH}}] = 10$ 步。两个模式**长度都是 3**、每个具体三连出现的概率都是 $\frac{1}{8}$ ，可平均等待却差了 4 步。差别的全部根源是**自重叠**（self-overlap）：HHH 自己跟自己高度重叠，一旦中途断掉要”退”得很远，所以等得久。

记住这个词**自重叠**。这一部分我们会看到：平均等待短 $\neq$ 对决胜率高，而自重叠正是”没有最强模式”这件怪事背后的发动机。

先给你一个会颠覆直觉的剧透

你可能觉得：HHH 和 THH 都是长度 3，对决应该差不多五五开吧？**错得离谱**。后面会严格算出 THH 以 **7:1** 碾压 HHH。为什么？因为只要序列里冒出任何一个 T，HHH 就基本没戏了——那个 T 立刻变成 THH 的开头。这个剧透你先记着，第 2 章会把它变成精确的 $\frac{1}{8} : \frac{7}{8}$ 。

第 2 章 可靠解：联合马尔可夫链

本章给出整篇讲义里唯一**从头到尾正确、对任意模式（包括不等长）都成立、而且被代码反复验证**的解法。后面我们会推一条”更漂亮”的自创公式，结果翻车了——到时候就靠本章这个解当**裁判**，判那条公式对不对。所以请把本章学扎实。

2.1 为什么状态要”两个进度一起记”

先想清楚：玩这个游戏，你脑子里要记住什么？

假设你正盯着序列一个一个往后看，目标是判断”甲（ X ）还是乙（ Y ）会先凑齐”。你脑子里真正需要记住的，不是前面已经出现过的一长串历史，而是**此刻两个模式各自’差几个字符就成了’**。

比如目标 $X = \text{HH}$ 、 $Y = \text{HTH}$ 。如果当前序列的结尾是 H，那么：对 $X = \text{HH}$ 来说，我已经垫了 1 个 H，再来一个 H 就成了（进度 1）；对 $Y = \text{HTH}$ 来说，我也垫了它的开头 H（进度 1）。这”**两个进度**”就是你需要记的全部——再往前的历史，对未来谁先赢没有任何影响。这就是数学里说的”无后效性”（马尔可夫性）。

于是我们把游戏的状态定义成一个**进度对**

$$s = (i, j), \quad \begin{cases} i = \text{当前结尾对 } X \text{ 前缀的最长匹配长度 (" } X \text{ 垫到第 } i \text{ 格")}, \\ j = \text{当前结尾对 } Y \text{ 前缀的最长匹配长度 (" } Y \text{ 垫到第 } j \text{ 格")}. \end{cases}$$

每读入一枚新硬币， i 和 j 同时按各自模式的规则更新。如果某一边的进度达到了自己的满长度（ $i = |X|$ 或 $j = |Y|$ ），那一边就赢了，游戏在这个**吸收状态**停下。

和单模式（第一部分）唯一的本质区别

第一部分等单个 HHH 时，状态只有“一个进度”（HHH 垫到第几位）。这里是**两个模式抢同一条流**，所以状态要“两个进度一起记” (i, j) ，而且它们**共用每一枚硬币**——同一个字符可能让甲进一步、也可能同时让乙进一步或退回。这个“共用、耦合”正是双模式比单模式难的地方，也是后面我们一旦只记一个进度就翻车的根源。

2.2 一个失配要“退回去”：KMP 的倒退

为什么进度会“倒退”，而不是直接归零

设目标 $X = \text{HHH}$ ，当前已经垫到进度 2（结尾是 HH）。这时来了个 T，HHT 显然没法接着凑 HHH——进度该归零。但换个目标 $X = \text{HH}$ 配 $Y = \text{HTH}$ ：若 Y 已垫到 HT（进度 2），来一个 H，正好凑成 HTH， Y 赢；但若来的是 T，变成 HTT，对 $Y = \text{HTH}$ 来说……结尾的 T 还能不能用？HTT 的结尾 T 不是 Y 的开头 H，所以进度退回 0。

关键在于：**失配时进度不一定归零，而是退到“当前结尾还能匹配模式开头多少”的那个长度**。这个“退到哪”由模式自己的结构决定，正是 KMP 字符串匹配里的“失配函数”。你现在不需要会证 KMP，只要接受：每个（当前进度，新字符）都唯一决定一个新进度，可以查表得到。

我们把这个“查表”记成两个转移函数 $\delta_X(i, c)$ 和 $\delta_Y(j, c)$ ：给定当前进度和新读入的字符 c ，返回新进度。读入 c 后，状态 (i, j) 变成 $(\delta_X(i, c), \delta_Y(j, c))$ 。若新进度触到满长度，则进入对应的吸收态。

2.3 动手算：HHH 对 THH

我们把剧透中的 7:1 真正算出来。设 $A = \text{HHH}$ （甲）， $B = \text{THH}$ （乙）。

2.3.1 第一步：列出所有“还没分胜负”的状态

用**当前结尾的有效后缀**给状态命名（它唯一决定进度对 (i, j) ，写起来更直观）。可达的瞬态有五个：

$$a_0 = (\emptyset), \quad a_1 = (\text{H}), \quad a_2 = (\text{HH}), \quad b_1 = (\text{T}), \quad b_2 = (\text{TH}),$$

再加两个吸收态 $A = (\text{HHH 赢})$ 、 $B = (\text{THH 赢})$ 。

为什么只有 5 个瞬态，而不是想象中那么多

你也许会担心状态爆炸。但注意：当前结尾**不可能同时**既深度匹配 HHH 又深度匹配 THH——它们开头就不同（一个 H 一个 T）。**很多“组合”压根不可能出现**，所以实际可达的状态很少。”哪些组合不可能”这件事，本身就是自重叠/互重叠在悄悄起作用——记住这个感觉，它后面会以 Conway 相关数的形式正式登场。

2.3.2 第二步：写下每个状态读 H / 读 T 去哪

一个一个推（这是体力活，但每一步都要自己确认）：

- $a_0 = (\emptyset)$ ：读 H $\rightarrow$ 结尾变 $H = a_1$ ；读 T $\rightarrow$ 结尾变 $T = b_1$ 。
- $a_1 = (H)$ ：读 H $\rightarrow HH = a_2$ ；读 T $\rightarrow$ 结尾 $T = b_1$ 。
- $a_2 = (HH)$ ：读 H $\rightarrow HHH$ ，甲赢 = A ；读 T $\rightarrow$ 结尾 $T = b_1$ 。
- $b_1 = (T)$ ：读 H $\rightarrow TH = b_2$ ；读 T $\rightarrow$ 结尾还是 $T = b_1$ （自环！）。
- $b_2 = (TH)$ ：读 H $\rightarrow THH$ ，乙赢 = B ；读 T $\rightarrow$ 结尾 $T = b_1$ 。

那个自环 $b_1 \xrightarrow{T} b_1$ 是整道题的灵魂

看 $b_1 = (T)$ 读到 T：结尾从 T 变成 TT，但对 $B = THH$ 来说有用的只是最后一个 T，所以还停在“垫了一个 T” $= b_1$ 。这个自环意味着：一旦你掉进 T 这个状态，连抛多少个 T 都赖在这儿不走，稳稳保持着 B 的开头。这就是为什么 $B = THH$ 这么强——它的开头 T 像个“粘性陷阱”，而对手 $A = HHH$ 一旦出 T 就万劫不复。

2.3.3 第三步：把“胜率”写成方程组

定义 $h_s = \mathbb{P}(\text{从状态 } s \text{ 出发，甲 } A \text{ 先赢})$ 。两个边界显然：

$$h_A = 1 \text{ (甲已经赢了)}, \quad h_B = 0 \text{ (乙赢了，甲没戏)}.$$

为什么方程里“没有 +1”——这是和第一部分最容易混的地方

第一部分算期望步数时，每走一步要 +1（因为你在数步数），方程是 $e_s = 1 + \frac{1}{2}e_{s'} + \frac{1}{2}e_{s''}$ 。这里我们算的是概率，不是步数！从 s 出发的胜率，等于“读 H 之后的胜率”和“读 T 之后的胜率”各占一半的加权平均，不需要也不能加 1：

$$h_s = \frac{1}{2} h_{(\text{读 H 去的状态})} + \frac{1}{2} h_{(\text{读 T 去的状态})}.$$

一句话：数步数有 +1，算概率没有 +1。把这两件事分清，是这一章不出错的关键。

逐条写出来（用第二步的转移）：

$$\begin{aligned} h_{b_1} &= \frac{1}{2} h_{b_2} + \frac{1}{2} h_{b_1} & (b_1 \text{ 读 H 去 } b_2, \text{ 读 T 自环回 } b_1) \\ h_{b_2} &= \frac{1}{2} h_B + \frac{1}{2} h_{b_1} = \frac{1}{2} \cdot 0 + \frac{1}{2} h_{b_1} & (b_2 \text{ 读 H 乙赢, 读 T 去 } b_1) \\ h_{a_2} &= \frac{1}{2} h_A + \frac{1}{2} h_{b_1} = \frac{1}{2} \cdot 1 + \frac{1}{2} h_{b_1} & (a_2 \text{ 读 H 甲赢, 读 T 去 } b_1) \\ h_{a_1} &= \frac{1}{2} h_{a_2} + \frac{1}{2} h_{b_1} & (a_1 \text{ 读 H 去 } a_2, \text{ 读 T 去 } b_1) \\ h_{a_0} &= \frac{1}{2} h_{a_1} + \frac{1}{2} h_{b_1} & (a_0 \text{ 读 H 去 } a_1, \text{ 读 T 去 } b_1) \end{aligned}$$

2.3.4 第四步：解方程

从第一条 $h_{b_1} = \frac{1}{2} h_{b_2} + \frac{1}{2} h_{b_1}$ ，两边减去 $\frac{1}{2} h_{b_1}$ ，得 $\frac{1}{2} h_{b_1} = \frac{1}{2} h_{b_2}$ ，即 $h_{b_1} = h_{b_2}$ 。代入第二条 $h_{b_2} = \frac{1}{2} h_{b_1}$ ，于是 $h_{b_1} = \frac{1}{2} h_{b_1}$ ，只能 $h_{b_1} = 0$ ，从而 $h_{b_2} = 0$ 。再往上：

$$h_{a_2} = \frac{1}{2} + \frac{1}{2} \cdot 0 = \frac{1}{2}, \quad h_{a_1} = \frac{1}{2} \cdot \frac{1}{2} + 0 = \frac{1}{4}, \quad h_{a_0} = \frac{1}{2} \cdot \frac{1}{4} + 0 = \frac{1}{8}.$$

起点是 a_0 （什么都还没垫），所以

$$\mathbb{P}(\text{HHH 先}) = h_{a_0} = \frac{1}{8}, \quad \mathbb{P}(\text{THH 先}) = \frac{7}{8}.$$

$h_{b_1} = h_{b_2} = 0$ 把剧透彻底解释清楚了

解出来 $h_{b_1} = h_{b_2} = 0$ 是惊人的：只要游戏走到“出现过一个 T”（掉进 b_1 ），甲 HHH 获胜的概率精确为 0。原因：甲要从此刻起连出三个 H 才赢，但在凑齐之前几乎必然先冒一个 T，而那个 T 立刻把局面送回 b_1 ，并喂给乙 THH 一个完美开头。所以甲唯一的赢法是开局就直接 HHH——概率恰好 $\frac{1}{8}$ 。 $\frac{1}{8}$ 不是巧合，是“必须一气呵成、否则永世不得翻身”的数学写照。

本章带走的两条铁律

(1) 胜率 $\neq$ 期望等待。 $\mathbb{E}[T_{\text{HHH}}] = 14$ 说明甲平均等得久；但对决里它是 $\frac{1}{8}$ 的惨败。”等得快慢”和”对决胜负”是两套东西，别混。

(2) 联合链永远可靠。把状态取成进度对 (i, j) 、写齐次方程 $h_s = \frac{1}{2}h_{s'} + \frac{1}{2}h_{s''}$ 、解线性方程组——这套方法对任意两个模式、不等长也照样成立。本讲义后面所有”漂亮公式”都要用它来验真伪。

第 3 章 翻车历程：试图找一个”闭式公式”的那条死路

第 2 章的联合链已经能解任何对决，但它对每一对模式都要重新列方程组。数学家的本能是：能不能有一个闭式（closed form）——给我任意 X, Y ，套一个公式直接出胜率？这正是著名的 Conway 相关数公式想干的事。

本章记录我们自己对概率定义出发、想把这个闭式从头推出的全过程。剧透：我们推岔了，走进一条死路，花了七八个小时，最后被一行代码证伪。但请耐心读完每一个弯路——它们是这份讲义真正的肉。每个弯路都对应一个极容易犯的概念错误。

3.1 弯路一： $P(\text{长度} = n) = \frac{1}{2^n}$ 吗？

我们想按”游戏在第几步结束”来拆分。第一个念头：

尝试

$$\mathbb{P}(X \text{ 先}) = \sum_n P(\text{游戏长度} = n) \cdot P(X \text{ 赢} \mid \text{长度} = n), \quad \text{其中 } P(\text{长度} = n) = \frac{1}{2^n}.$$

翻车：把”一条路径的概率”当成”一个事件的概率”

$\frac{1}{2^n}$ 是某一条具体长度- n 序列的概率。但”游戏在第 n 步结束”是一整族序列组成的事件——有很多条不同的长度- n 序列都让游戏恰好在第 n 步结束。这个事件的概率是（这样的序列条数） $\times \frac{1}{2^n}$ ，绝不是 $\frac{1}{2^n}$ 。

这正是第一部分反复踩的”具体串 vs 事件”坑的换皮复发：钉死一个具体串，概率是 $\frac{1}{2^{\text{长度}}}$ ；但一个由很多串组成的事件，概率要把它们加起来。

想通：两种”样本空间”建模都对，但要选清楚

其实有两种合法的建模方式：

- 无限流模型：样本空间是所有无限序列 $\{H, T\}^\infty$ 。” X 在第 n 位首胜”是个无条件事件 E_n^X ，它的概率 = $\frac{\text{满足条件的有限前缀条数}}{2^n}$ （后面的无限尾巴自动约掉）。这里没有 $P(n)$ 这个因子。

· **按长度模型**：先选长度，再在该长度里算条件概率，这时 $P(n) = \frac{1}{2^n}$ 跑不掉。
两种算的是同一个联合概率 $\mathbb{P}(\text{停在 } n \text{ 且 } X \text{ 赢})$ ，只是一个写成”联合”、一个写成”边
际 $\times$ 条件”，**数值相等**。我们选第一种（无限流），于是干脆

$$\mathbb{P}(X \text{ 先}) = \sum_{n \geq |X|} \mathbb{P}(E_n^X), \quad E_n^X = \{T_X = n < T_Y\}.$$

这一步是**对的**，后面所有推导都从它长出来——连最后 GPT 的正解也保留了这一层。

3.2 弯路二：生存量漏掉了对手 Y

确定了 $\mathbb{P}(X \text{ 先}) = \sum_n \mathbb{P}(E_n^X)$ ，下一步要算单项 $\mathbb{P}(E_n^X)$ 。我们想照搬第一部分 单模式 HHH 那个漂亮递推 $P_n = \frac{1}{16}(1 - \sum_{k < n} P_k)$ 。

尝试

$\mathbb{P}(E_n^X) = (\text{收尾因子}) \times (1 - \sum_{k < n} \mathbb{P}(E_k^X))$ ，那个 $1 - \sum \mathbb{P}(E_k^X)$ 当作”前面还没出现 X ”的生存概率。

翻车：单模式的”生存”在双模式里漏了一半

$1 - \sum_{k < n} \mathbb{P}(E_k^X)$ 只是” X 还没赢”。但 E_n^X 要求的是”在第 n 步之前 X 和 Y 都没赢”。这个表达**漏掉了** Y ：它没有扣掉” Y 已经先赢、游戏其实早结束了”的那些情形。

为什么不能像单模式那样简单？因为现在的”生存”是**联合生存**——要 X 和 Y 同时都没出现。而联合生存**不能拆成**” X 没出现的概率 $\times$ Y 没出现的概率”，因为它们共用同一条硬币流，高度耦合，不独立。

直觉：耦合到底意味着什么

想象两个事件”前 10 步没出现 HHH”和”前 10 步没出现 THH”。它们用的是同一串硬币——如果这串硬币里有很多 T，那 HHH 更难出现（好事）、THH 更容易出现（坏事）。两个事件被同一串硬币**反向牵连**，所以绝不独立。这就是为什么双模式问题的状态必须把两个进度绑在一起记——又一次指向 (i, j) 联合状态。

3.3 弯路三： (k, p) 划分不互斥（”相容”陷阱）

我们退回去想正确的拆分。要算 $\mathbb{P}(E_n^X)$ ，自然想到”按 Y 在前面垫了多少”来进一步划分。

尝试

用两个指标 (k, p) 划分： $k = Y$ 匹配了前几位， $p =$ 这段匹配发生在哪个位置。然后 $\mathbb{P}(E_n^X) = \sum_k \sum_p \mathbb{P}(E_n^X, Y \text{ 的前 } k \text{ 位在位置 } p \text{ 出现})$ 。

翻车：这些事件”相容”，求和会重复计数

一条序列里， Y 的部分前缀可以在好几个不同的 (k, p) 同时出现。举例：序列 THT，对 $Y = \text{THH}$ ，它在位置 1 有”T”（ $k=1, p=1$ ）、位置 3 又有”T”（ $k=1, p=3$ ）、位置 1-2 有”TH”（ $k=2, p=1$ ）……同一条序列被好几个 (k, p) 项同时数到。

术语上：这些 (k, p) 事件是**相容**（compatible，能同时发生）的，**不是互斥**（mutually exclusive）。而概率的可加性 $\mathbb{P}(\bigcup E_i) = \sum \mathbb{P}(E_i)$ 只对互斥事件成立。所以 $\sum_{k,p}$ 会重复

计数，根本不等于 $\mathbb{P}(E_n^X)$ 。

互斥 vs 相容：用最直白的话区分

互斥：两件事不可能同时发生（掷骰子”出 1 点”和”出 2 点”）。它们的概率可以直接相加。**相容**：两件事可以同时发生（”出偶数”和”出大于 3”，因为 4、6 同时满足）。直接相加会把重叠部分算两遍。我们的 (k, p) 就是相容的——一条序列前面可以有好几处 Y 的零碎匹配，于是被反复计入。

想通：改用”此刻的唯一进度”，它天然互斥

要让划分互斥，第二层指标必须是**某个对每条序列只有唯一取值**的量。那个量就是：进入收尾那一刻， Y 当前的匹配进度。记 $m = n - |X|$ （收尾段开始前的位置），定义

$$\Gamma_m = \max\{j : 0 \leq j \leq |Y| - 1, \omega_{m-j+1..m} = Y_{1..j}\}.$$

即”前段末尾 j 个字符恰好等于 Y 的前 j 位”的**最大 j** 。一条序列的 Γ_m **唯一**（最长匹配长度只有一个值），所以按 $\Gamma_m = j$ 划分是**互斥**的真等价关系：

$$\mathbb{P}(E_n^X) = \sum_{j=0}^{|Y|-1} \mathbb{P}(E_n^X, \Gamma_m = j).$$

关键差别一句话：” Y 的前缀历史上在哪儿出现过”（多个位置，相容）vs ” Y 此刻进度多少”（唯一值，互斥）。前者错，后者对。马尔可夫状态的精髓正是丢掉历史、只留此刻进度。

3.4 弯路四：一条”看起来很漂亮”的自创递归

有了互斥的 Γ_m ，我们搭出一条自创推导，当时觉得离 Conway 闭式只差临门一脚：

1. **第一层（沿时间）**： $\mathbb{P}(X \text{ 先}) = \sum_n \mathbb{P}(E_n^X)$ ，按 X 首胜位置，互斥。✓
2. **第二层（沿进度）**： $\mathbb{P}(E_n^X) = \sum_j \mathbb{P}(E_n^X, \Gamma_m = j)$ ，按 Γ_m ，互斥。
3. **分段**： $m = n - |X|$ ；前段 $1..m$ ；收尾段 $m + 1..n$ 钉死为 X ，贡献因子 $\frac{1}{2^{|X|}}$ 。
4. **角色轮换**（这一步是后面翻车的祸根）：把”前段以 Y 的前 j 位 $Y_{1..j}$ 结尾”看成一个同型子问题——残留前缀 $Y_{1..j}$ **升格成新的”被追逐目标”**，原模式 X **降为对手**。于是得到自指递归

$$V(X, Y) = \frac{1}{2^{|X|}} \sum_{j=0}^{|Y|-1} \chi_j(X, Y) V(Y_{1..j}, X), \quad \chi_j(X, Y) = \mathbb{1}[X_{|X|-j+1..|X|} = Y_{1..j}].$$

其中 χ_j 是” X 的后 j 位 = Y 的前 j 位”的重合指示。

当时为什么觉得它对——那种”统一感”是真实的

顺着这条递归走，我们自己撞见了好几样东西：递归会层层嵌套出一个不变量； X 、 Y 主客易位像一场博弈的先后手； χ_j 那些重合项看起来正是相关数 $L(X, Y)$ 的雏形……当时强烈感到”这背后有个守恒的、赌博式的结构”，也就是**鞅的味道**。这种”从自己的推导里闻到统一”的体验是珍贵的、真实的——即使这条具体递归最后被证明是错的。学数学，这种”差点摸到”的感觉本身就是养分。

3.5 弯路五：Claude 凭记忆给出一个假公式，还说“很明显”

为了把上面的 χ_j 求和接到“教科书里的经典 Conway 公式”，Claude（就是写这份讲义的 AI）凭训练记忆给出了一套相关数 L 的定义，和一个四项差值公式

$$\frac{\mathbb{P}(A\text{先})}{\mathbb{P}(B\text{先})} \stackrel{?}{=} \frac{L(B, B) - L(B, A)}{L(A, A) - L(A, B)},$$

并断言一个恒等式 $K(A, B) = L(B, B) - L(B, A)$ “很明显”。

翻车：这套定义和“很明显”全是错的

Claude 给的 L 定义权重方向、求和范围、分子分母顺序多处记错，那个“很明显”的恒等式更是根本不成立的假命题。这不是当场推导出的结论，而是训练数据里的模糊记忆被当成了事实——AI 的典型毛病：把“我印象里好像是这样”当成“我证明了这样”。

GPT 用一个两字符反例当场打脸

我们把这个“待证恒等式”丢给另一个 AI（GPT）。它没有顺着推，而是直接找反例：取最短的 $A = \text{HH}$ 、 $B = \text{HT}$ ，逐项算：

$$K(\text{HH}, \text{HT}) = 3, \quad L(\text{HT}, \text{HT}) - L(\text{HT}, \text{HH}) = 2 - 0 = 2.$$

$3 \neq 2$ 。那个“很明显”的恒等式当场被证伪。而 HHH/THH 那里两边碰巧都等于某个值、看起来吻合，纯属巧合，正是它给了 Claude 虚假的信心。记忆驱动的“声称” vs 反例驱动的“证伪”——后者完胜。

方法论提醒（第一次，后面还会强化）

当有人（包括 AI、包括你自己）说某个公式“很明显”、却不当场证明时，最有效的回应不是相信，而是找反例——尤其挑最小、最特殊的例子（这里是长度 2）。一个反例就能推翻一个“显然”。这就是为什么我们要养成“用具体小例子 + 代码验证”的习惯，而不是迷信“看起来对”。

第 4 章 验证危机：代码裁判一锤定音

上一章我们已经知道那个“四项 L 公式”被反例打脸了。但我们自己推的那条递归 $V(X, Y) = \frac{1}{2^{|X|}} \sum_j \chi_j V(Y_{1..j}, X)$ 呢？它在 HHH/THH 上验证过给出正确的 $\frac{1}{7}$ 赔率，所以我们以为递归本身是对的，只是接 L 的方式记错了。这个以为，也错了。

4.1 用“裁判”检验：不等长的例子

为什么必须换一个不等长的例子

HHH 和 THH 都是长度 3、而且自重叠结构非常特殊（一个极强、一个极弱）。很多错误公式在这种极端例子上会碰巧给对。要真正检验一个公式，必须用结构不同的例子——特别是不等长的，因为我们的推导从头就允许不等长，这正是最容易暴露问题的维度。

取 $A = \text{HH}$ （长 2）、 $B = \text{HTH}$ （长 3）。先用第 2 章的联合链（裁判）算真值，再用我们的递归算，对比。

4.1.1 裁判给出真值

联合链（代码精确求解，并用蒙特卡洛独立复核）给出：

$$\mathbb{P}(\text{HH 先于 HTH}) = \frac{2}{3}, \quad \text{赔率} = 2 : 1.$$

4.1.2 我们的递归给出什么

照着递归一步步手算（这是体力活，但每步都能查）：

$$V(\text{HH}, \text{HTH}) = \frac{1}{2^2} \left[\chi_0 V(\emptyset, \text{HH}) + \chi_1 V(\text{H}, \text{HH}) + \chi_2 V(\text{HT}, \text{HH}) \right].$$

其中 $\chi_0 = 1$; $\chi_1 = \mathbb{P}[\text{H} = \text{H}] = 1$; $\chi_2 = \mathbb{P}[\text{HH} = \text{HT}] = 0$ 。又 $V(\emptyset, \text{HH}) = 1$ ，且

$$V(\text{H}, \text{HH}) = \frac{1}{2} [V(\emptyset, \text{H}) + V(\text{H}, \text{H})] = \frac{1}{2} \left[1 + \frac{1}{2} \right] = \frac{3}{4}.$$

代回：

$$V(\text{HH}, \text{HTH}) = \frac{1}{4} \left[1 + \frac{3}{4} \right] = \frac{7}{16} = 0.4375.$$

翻车：递归给 $\frac{7}{16}$ ，真值 $\frac{2}{3}$ ——递归本身就错了

$\frac{7}{16} = 0.4375 \neq 0.6667 = \frac{2}{3}$ 。差得很远。这说明问题不在“接 L 的最后一步”，而在递归本身。我们以为 HHH/THH 验证过 $\frac{1}{8}$ 就万事大吉——但那一对太特殊，又一次碰巧给对，骗了我们整整一晚上。

唯一确定的硬事实——以后都以它为准

三个完全独立的方法（联合链解线性方程、蒙特卡洛模拟、以及后面会讲的矩阵闭式）一致给出

$$\mathbb{P}(\text{HH 先于 HTH}) = \frac{2}{3}.$$

任何声称解出这道题的公式或递归，都必须复现这个 $\frac{2}{3}$ 。这就是我们的“试金石”。凡是在 HHH/THH 上对、却复现不出 HH vs HTH 的 $\frac{2}{3}$ 的，一律判错。

第5章 正解：局面、矩阵闭式与鞅

GPT 审阅了我们的整条推导，精确指出了两处实质错误，并给出修正版。修正版的矩阵闭式已被代码验证对等长、不等长全部正确（ $\frac{1}{8}, \frac{7}{8}, \frac{2}{3}, \frac{1}{3} \dots$ 都对）。这一章把它讲透。

5.1 第一处错：只记 $\Gamma_m = j$ 不够，要记完整局面

问题出在“跨边界”那一刻

回忆我们的分段：前段 $1..m$ ，收尾段 $m+1..n$ 钉死为 X 。我们以为只要记住“进入收尾时 Y 垫了 j 位”（ $\Gamma_m = j$ ）就够了。但想象一下：前段的尾巴 + 收尾段 X 的前几个字符拼接起来，可能在走到第 n 位之前就提前凑成了 X 或 Y ！

要判断“会不会提前凑成 X ”，光知道 Y 的进度 j 没用——你还得知道当前尾巴对 X 垫了多少（进度 i ）。所以足够的状态必须是两个进度一起：局面 (i, j) 。

根本病因： $\Gamma_m = j$ 是”单边进度”，丢了 X 那一维

$\Gamma_m = j$ 只记了 Y 的进度。但收尾段是 X 的字符，它在拼接处既可能帮 Y 续命、也可能让 X 提前完成。要正确处理”跨边界提前终止”，状态必须同时记 X 进度 i 和 Y 进度 j 。一句话：**足够状态是局面 (i, j) ，不是单边的 $\Gamma_m = j$ 。**——而这个 (i, j) ，正是第 2 章联合链的状态。

5.2 第二处错：角色轮换把”进度”误当成”新模式”

根本病因： $Y_{1..j}$ 不是新串，是 Y 的进度状态

我们的递归把” Y 已垫到 j ”写成子问题 $V(Y_{1..j}, X)$ ，把残留前缀 $Y_{1..j}$ 当成了一个全新的短模式从头追。但”完整短串 $Y_{1..j}$ 从零开始追”和” Y 已经垫了 j 位、继续往 $|Y|$ 推”不是同一回事——后者带着”已匹配 j 位”的记忆，它失配时退到哪、能否续接，都依赖这个记忆。

我们用”两个完整字符串”当递归参数，丢掉了马尔可夫该保留的进度记忆。这就是为什么这条递归必然崩——它在每一次”轮换”时都悄悄遗忘了一部分状态信息。

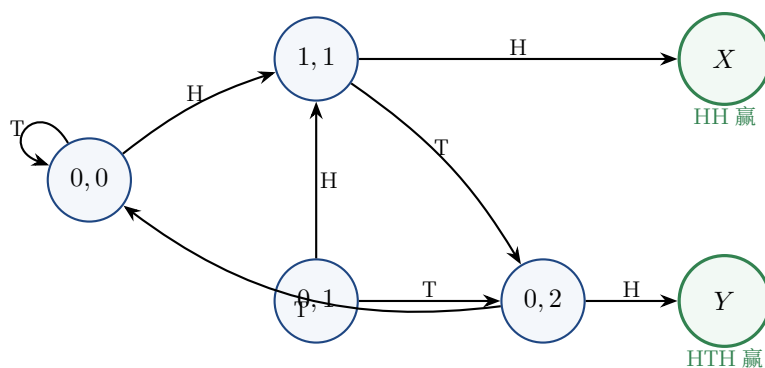
想通：正确状态一直是 (i, j) ，我们绕回了起点

正确的递归状态必须是**进度局面 (i, j)** ——这恰恰就是第 2 章那个联合马尔可夫链。我们花一整晚自创的”两完整串递归”，绕了一大圈，被 GPT 一句话送回起点：**联合链一直是对的，因为它的状态保留双进度记忆；我们的递归丢了记忆，所以错。**

注意：错不在”两层划分”那个大思路（那个是对的、有价值的，正解也保留了它），错在**把状态偷偷退化成了”完整串”**。这是一个极其微妙、极其值得记住的滑动：同样叫”递归”，状态选 (i, j) 就对，选”完整串”就错。

5.3 看一张图：HH 对 HTH 的联合链

为了把”局面”讲具体，我们画出试金石那一对 $X = \text{HH}$ 、 $Y = \text{HTH}$ 的联合链。状态写成 (i, j) ， i 是 HH 的进度、 j 是 HTH 的进度。从起点 $(0, 0)$ 可达的瞬态有四个，加两个吸收态。



(从 $(0, 0)$ 出发，状态 $(1, 0)$ 与 $(1, 2)$ 不可达，已略去。) 读法：实线箭头上的 H/T 表示读到该字符后的走向；走到 X 表示 HH 先凑成、走到 Y 表示 HTH 先凑成。

从图上”看出” $\frac{2}{3}$ 的直觉

看状态 $(1, 1)$ ：读 H 直接 X 赢。再看 $(0, 0)$ 读 H 就进 $(1, 1)$ 。所以”开头两个 H”基本就锁定 HH 赢了。而 HTH 要赢，必须先走到 $(0, 2)$ （即出现 HT）再来个 H，路径更绕。直觉上 HH 占优，精确算出来正是 $\frac{2}{3}$ 。

5.4 矩阵闭式：一个公式搞定等长与不等长

现在把”按局面划分”写成线性代数。设 S 为所有非吸收局面的集合。

5.4.1 安全转移矩阵 M

M 在数什么

M 记录”从一个局面，读一枚硬币，在不分胜负的前提下能走到哪个局面”。因为每个局面读 H、读 T 各一条路，所以从 s 到 t 的”安全走法”条数可能是 0、1 或 2。

$$M_{s,t} = \#\{a \in \{H, T\} : s \xrightarrow{a} t \text{ 且这一步没有任何一方获胜}\} \in \{0, 1, 2\}.$$

真实的概率转移是 $M/2$ （每个字符 $\frac{1}{2}$ ）。对试金石 HH vs HTH，把状态按 $(0,0), (0,1), (0,2), (1,0), (1,1), (1,2)$ 排序，安全转移矩阵是

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

（例如第一行： $(0,0)$ 读 T 回到 $(0,0)$ 贡献对角的 1，读 H 去 $(1,1)$ 贡献第 5 列的 1。）

5.4.2 前段计数： $N_m = e_0 M^m$

为什么 M 的幂在数”安全前段”

M 是”走一步且不分胜负”的计数。那么 M^m 的某个元素，就是”安全地走 m 步、从起点到达某局面”的路径条数。配上每条路径的概率 2^{-m} ，就得到”长度- m 的安全前段停在各局面”的概率。

设 e_0 是起点局面 $(0,0)$ 的单位行向量。则”长度- m 、全程不分胜负、停在各局面”的路径条数向量是

$$N_m = e_0 M^m.$$

5.4.3 最后冲刺筛选 R_X ：替换错误的 χ_j

为什么 χ_j 不够，要换成”动态筛选”

我们原来的 χ_j 只是一个静态的 overlap 指示（” X 后缀是否等于 Y 前缀”）。但真正需要判断的是：从某个局面 s 出发，强行把整段 X 接上去，会不会中途就提前凑成 X 或 Y ？只有”中途都安全、最后一位才由 X 收尾”才算合法。这需要沿整段 X 逐位动态检查，不是一个静态 overlap 能表达的。

定义筛选向量 $R_X(s) \in \{0, 1\}$ ：

$$R_X(s) = 1 \iff \text{从 } s \text{ 强行读入 } x_1 x_2 \cdots x_{|X|}, \text{ 前 } |X|-1 \text{ 步不分胜负, 最后一位恰由 } X \text{ 吸收.}$$

对试金石那一对，算出 $R_X = (1, 1, 0, 0, 0, 0)^\top$ （只有局面 $(0,0)$ 和 $(0,1)$ 接 HH 是合法冲刺）。

5.4.4 合起来：闭式

把”安全前段计数 × 收尾因子 × 冲刺筛选” 对所有 m 求和：

$$\mathbb{P}(E_{m+|X|}^X) = 2^{-(m+|X|)} e_0 M^m R_X \Rightarrow \mathbb{P}(X \text{ 先}) = \sum_{m \geq 0} 2^{-(m+|X|)} e_0 M^m R_X.$$

把 $2^{-|X|}$ 提出来，里面是几何级数 $\sum_m (M/2)^m$ 。因为游戏迟早分胜负，这个级数收敛到 $(I - \frac{M}{2})^{-1}$ ：

$$\mathbb{P}(X \text{ 先}) = 2^{-|X|} e_0 \left(I - \frac{M}{2}\right)^{-1} R_X$$

对称地 $\mathbb{P}(Y \text{ 先}) = 2^{-|Y|} e_0 (I - \frac{M}{2})^{-1} R_Y$ 。

代码验证：这个闭式真的对

对试金石 $X = \text{HH}$ 、 $Y = \text{HTH}$ ，代入算出

$$\mathbb{P}(\text{HH 先}) = 2^{-2} e_0 \left(I - \frac{M}{2}\right)^{-1} R_X = \frac{2}{3}. \quad \checkmark$$

$((I - \frac{M}{2})^{-1})$ 的第一行是 $(\frac{8}{3}, 0, \frac{2}{3}, 0, \frac{4}{3}, 0)$ ，点乘 $R_X = (1, 1, 0, 0, 0, 0)^\top$ 得 $\frac{8}{3}$ ，再乘 $2^{-2} = \frac{1}{4}$ 得 $\frac{2}{3}$ 。我们用代码对 HHH/THH 、 HH/HTH 等多对验证，全部正确——包括不等长。这才是真正的闭式，而且它解释了我们的 χ_j 错在哪：应该是动态的 R_X ，不是静态的 χ_j 。

5.5 鞅到底在哪里

这是 GPT 比我们之前深入的地方。我们之前在”该不该追求鞅” ”Conway 是不是给鞅披了层皮” 上反复纠结，始终没把鞅精确定位。下面讲清楚。

5.5.1 三个极易混淆的对象

先用一句话区分三者

R_X 是”能不能合法冲刺”的 0/1 判断； $W(s)$ 是”未来重新完整出现一段 X ”的概率； $F(s)$ 才是”从此刻起 X 最终先赢”的真实概率。三者层层不同，混了就全乱。

- $R_X(s)$ ：0/1 筛选器，只回答”从 s 接整段 X 是否合法冲线”。不是概率。
- $W(s) = \sum_m 2^{-(m+|X|)} e_s M^m R_X$ ：从 s 出发，未来重新完整出现一段 X 作为终局块的概率和。它漏了”短补齐”：若当前局面 s 已经垫了 X 的一部分（比如 $X = \text{HH}$ 、 s 已垫一个 H，下一个 H 立刻就赢），这条路径属于真实获胜，却不是”未来重新完整出现一整段 X ”。所以 $F(s) = B(s) + W(s)$ ， $B(s)$ 是这些短补齐项。
- $F(s) = \mathbb{P}_s(T_X < T_Y)$ ：从局面 s 出发 X 最终先赢的真实吸收概率。这才是值函数，也就是第 2 章的 h_s 。

5.5.2 F 满足 Bellman 方程，且 $F(S_{t \wedge \tau})$ 是严格鞅

对任意非吸收局面 s ，按下一枚硬币分支（直接让 X 赢贡献 1、让 Y 赢贡献 0、进入非吸收 t 贡献 $F(t)$ ）：

$$F(s) = b_X(s) + \frac{1}{2} \sum_t M_{s,t} F(t), \quad b_X(s) = \frac{1}{2} \# \{a : s \xrightarrow{a} X \text{ 吸收}\},$$

边界 $F(X \text{ 赢}) = 1, F(Y \text{ 赢}) = 0$ 。

这就是第一部分那个“一步分析”，只是升级到双模式

这条方程的意思是：“此刻的胜率 = 下一枚硬币各种走向的胜率的平均”。它和第一部分等待 HHH 时的一步分析（first-step analysis）是同一个东西，也就是 **Bellman 期望方程**（动态规划里的策略评估）。注意它**没有** max——因为硬币是随机的，没有“决策”，所以是纯期望、不是最优化。

令 S_t 为第 t 步局面， $\tau = T_X \wedge T_Y$ （谁先赢的时刻）。Bellman 方程恰好说明

$$\mathbb{E}[F(S_{(t+1) \wedge \tau}) \mid S_{t \wedge \tau} = s] = F(s) \quad \Rightarrow \quad F(S_{t \wedge \tau}) \text{ 是严格鞅.}$$

鞅是什么，用赌博的话说

鞅就是一个“公平游戏的当前身价”：不管现在走到哪一步，下一步身价的期望恰好等于当前身价（不赚不亏）。这里 $F(s)$ 就是“当前局面下 X 最终赢的概率”，它正好有这个性质：再抛一枚硬币，胜率平均不变（虽然具体往上或往下跳）。**注意：是条件均值不变，不是每条路径数值不变**——单条路径上 $F(S_t)$ 会上下跳，但期望钉死。

5.5.3 odds 是两个鞅初值之比

设 $F_X(s) = \mathbb{P}_s(T_X < T_Y)$ 、 $F_Y(s) = \mathbb{P}_s(T_Y < T_X)$ ，两者沿 $S_{t \wedge \tau}$ 都是鞅，终点分别是“ X 赢”和“ Y 赢”的指示。于是

$$\frac{\mathbb{P}(X \text{ 先})}{\mathbb{P}(Y \text{ 先})} = \frac{F_X(s_0)}{F_Y(s_0)}.$$

关键澄清：是“初值之比”，不是“过程之比”

$\frac{F_X(S_t)}{F_Y(S_t)}$ 一般不是鞅。正确说法是：odds 等于两个鞅在初始局面 s_0 的取值之比。这个区别很容易搞错——我们之前“两赌徒 vs 每步进新赌徒”的纠结，根子就在没分清“鞅过程”和“鞅初值”。

回看我们之前所有关于鞅的纠结

“理性赌博该追求鞅吗”“Conway 是不是给鞅披了层皮”——现在彻底清楚了。鞅不是我们之前盯的“赌资 $W(s)$ ”或“筛选 R_X ”，而是**吸收概率值函数 $F(s)$** （就是联合链的 h_s ）。它是 Bellman 方程的解， $F(S_{t \wedge \tau})$ 严格鞅。等长情形的 Conway 公式，正是**这个鞅结构在“重叠数”上坍塌的结果**——鞅不是皮，它是“把整个局面 DP 压缩成 overlap 差值”的那台机器。我们之前一直“统一感建立不起来”，正是因为**盯错了对象**（盯着 W/R_X 而不是 F ）。

第6章 三种情形与那个漂亮的 Conway 公式

矩阵闭式 $2^{-|X|} e_0(I - \frac{M}{2})^{-1} R_X$ 对一切情形都成立。但根据 X, Y 长度关系，可以分三档，其中等长那档会坍塌成一个特别漂亮的公式。

6.1 情形一：两模式完全相同

若 $X = Y$ ，则 $T_X = T_Y$ ，“ X 严格先于 Y ”根本不可能发生， $\mathbb{P}(T_X < T_Y) = 0$ 。所以 Penney 默认讨论 $X \neq Y$ 。

6.2 情形二：等长但不同——Conway 漂亮公式

为什么等长能“坍缩”成简单公式

当 $|X| = |Y| = n$ 时，“提前跨边界终止”这种麻烦事不会破坏对称性，矩阵逆 $(I - \frac{M}{2})^{-1}$ 的作用可以化简成纯粹的“重叠数”运算。结果就是 Conway 那个只用自重叠和互重叠就能写出的公式。

定义 **Conway 相关数** (correlation number)：对两个长度都为 n 的串 U, V ,

$$C(U, V) = \sum_{k=1}^n \mathbb{1}[U \text{ 的后 } k \text{ 位} = V \text{ 的前 } k \text{ 位}] \cdot 2^{k-1}.$$

$C(U, U)$ 叫**自重叠** (U 的后缀和自己前缀有多重合)， $C(U, V)$ 叫**互重叠**。则赔率

$$\frac{\mathbb{P}(X \text{ 先})}{\mathbb{P}(Y \text{ 先})} = \frac{C(Y, Y) - C(Y, X)}{C(X, X) - C(X, Y)}$$

直觉：分子分母在量什么

分母 $C(X, X) - C(X, Y) =$ “ X 跟自己有多像” 减 “ X 跟对手有多像”。自重叠越大 (X 越容易被自己的尾巴卡住、越难凑齐)，它越吃亏；跟对手重叠越大 (X 的尾巴越容易喂给对手)，它也越吃亏。这个“自相似减互相似”的差，精确刻画了一个模式的强弱——这正是第 2 章“自重叠是发动机”那句话的代数化身。

验证 ($A = \text{HHH}$, $B = \text{THH}$, 权重 2^{k-1} : $k=1:1, k=2:2, k=3:4$) :

$$\begin{aligned} C(A, A) &= C(\text{HHH}, \text{HHH}) = 1 + 2 + 4 = 7, \\ C(B, B) &= C(\text{THH}, \text{THH}) = \underbrace{0}_{k=1} + \underbrace{0}_{k=2} + \underbrace{4}_{k=3} = 4, \\ C(A, B) &= C(\text{HHH}, \text{THH}) = 0 \quad (\text{H 后缀从不等于 T 开头的 } B \text{ 前缀}), \\ C(B, A) &= C(\text{THH}, \text{HHH}) = \underbrace{1}_{k=1} + \underbrace{2}_{k=2} + \underbrace{0}_{k=3} = 3. \end{aligned}$$

$$\frac{\mathbb{P}(A \text{ 先})}{\mathbb{P}(B \text{ 先})} = \frac{C(B, B) - C(B, A)}{C(A, A) - C(A, B)} = \frac{4 - 3}{7 - 0} = \frac{1}{7} \Rightarrow \frac{1}{8} : \frac{7}{8}. \quad \checkmark$$

和联合链完全一致 (代码验证)。

6.3 情形三：不等长——不能硬套 Conway 差值

这正是我们整晚翻车的总根源

当 $|X| \neq |Y|$ 时，不能套等长 Conway 差值公式。最极端的例子： $X = \text{HH}$ 、 $Y = \text{HHH}$ 。只要 HHH 出现，HH 必然已经提前出现了，所以 Y 根本不可能严格先于 X ($\mathbb{P}(Y \text{ 先}) = 0$)。这种“一个是另一个的前缀/子串”的纠缠，等长公式完全处理不了。

通用解只能是矩阵闭式 $2^{-|X|} e_0 (I - \frac{M}{2})^{-1} R_X$ ，等长时它才坍缩成 Conway。而我们的整条自创推导从头就允许不等长 (这是对的、是更一般的)，可 Claude 凭记忆搬来的“经典 Conway 公式”只对等长——用等长的尺子量不等长的布，双重错位，这就是七八个小时翻车的总根源。

一个关键的人为警觉救了全场

是 Bo 反复坚持“我们推的是不等长”这一点，才最终捅破假象——它逼出了“等长 Conway 不能套 不等长”这个区分。在一群都在自信输出的来源（包括 AI 的记忆）面前，坚持一个看似不起眼的结构性事实（长度可不等），往往就是那根戳破气球的针。

6.4 顺便：那个被丢弃的 K 和真 Conway C 的关系

- 我们自创的 $K(X, Y) = \sum_{k=0}^{|Y|-1} c_k 2^k$ （带 $k=0$ 空对齐项）：作为一个 overlap 分数可以定义，但 $\frac{K(A, B)}{K(B, A)}$ 不是正确赔率，连等长也不总对（HH vs HT 即反例： K 给 1:1，真值另有其数）。**丢弃。**
- 真 Conway C （权重 2^{k-1} 、 k 从 1、差值形式）：等长正确，代码验证。
- 不等长：没有简单的二项 overlap 比值，老老实实用矩阵闭式。

第 7 章 回到问题二：没有最强模式（非传递性）

现在回答开头的问题二。用任一正确工具（联合链 / 矩阵闭式 / 等长 Conway）两两计算长度-3 模式，会发现一个惊人的循环：

$$HHH \prec THH \prec TTH \prec HTT \prec HHH,$$

每个“ $\prec$ ”表示“后者以 $\geq 2:1$ 击败前者”。它首尾相接成一个圈——像石头剪刀布，没有谁是最强的。

所以“挑期望等待最短的”不是必胜策略

这是一个 **minimax / 非传递** 结构：无论你先选哪个模式，对手都能在看到你的选择后，挑一个专门克你的模式（直觉上：把“几乎是你模式开头”的字符接在你前面，截胡你）。所以“先选”反而吃亏，没有“天下无敌”的模式。

非传递性的发动机，还是**自重叠的不对称**——第一部分那个 $\mathbb{E}[T_{HHH}]=14$ vs $\mathbb{E}[T_{HTH}]=10$ 的差异，在这里长成了“谁也不是最强”的循环。一条线索（自重叠）贯穿第一、第二部分始终。

为什么这件事如此反直觉、又如此重要

我们的大脑默认“强弱可以排成一条线”（传递性）：若甲胜乙、乙胜丙，那甲必胜丙。Penney's game 是一个干净利落的**反例**：胜负关系可以成环。这种非传递性在现实里到处都是——选举里的投票悖论、生态里的“石头剪刀布”物种竞争、甚至某些骰子游戏。Penney 用最简单的硬币把它演示得明明白白。

第 8 章 元教训：当来源都会犯错，如何逼近真相

这一整段翻车，是一次多 AI 互相拆台 + 代码裁判的完整实战。对你（未来要做研究、要用 AI 工具的人）而言，这套方法论比 Conway 公式本身更值钱。

8.1 完整的“翻车—救回”链条

1. Claude 凭记忆给错 L 定义 $\rightarrow$ Bo 出于信任，花半天去证一个本就不成立的恒等式。

2. GPT 用 HH vs HT 两字符反例，当场证伪那个假恒等式。
3. 我们自推的 K 比值版” 在 HHH/THH 上验证过 $\frac{1}{7}$ ”，给了虚假信心。
4. 代码联合链（裁判）用 HH vs HTH 不等长例子，一锤定音： K 版错、递归本身也错、真值是 $\frac{2}{3}$ 。
5. GPT 审稿定位两处实质错误（状态不足、角色轮换），给出修正的矩阵闭式与鞅。
6. Claude 再用代码验证 GPT 的矩阵闭式（不靠信任，靠运行），确认 $\frac{1}{8}, \frac{2}{3}$ 全对，才敢写进这份讲义。

8.2 四条可迁移的纪律

把这四条刻进脑子

- (1) 没有哪个来源单独可信——包括 AI、包括”经典公式”、包括你自己的直觉。AI 有”记忆偏差”，会把”印象里好像是”当成”我证明了”。凡是”很明显””经典公式就是这样”而不当场推导/不查证的，默认怀疑。
- (2) 单点验证会给假信心。一个例子对 $\neq$ 公式对。HHH/THH 太特殊，让两个错误公式都碰巧蒙对。要用多个、结构不同、尤其不等长的例子去测。
- (3) 代码（或可独立计算的 ground truth）是最终裁判。漂亮的自创推导、记忆中的经典公式、”看起来统一”的感觉，都可能骗你；能跑出数的东西不会骗你。养成”推完就写个小程序验一验”的习惯。
- (4) 交叉拆台胜过单点深挖。让不同来源（Claude / GPT / DeepSeek / 代码）互相找茬，真相会在碰撞中浮出来。本场就是 GPT 拆 Claude 的台、代码拆所有公式的台，最后逼出正解。

8.3 真正沉淀下来的数学

这份讲义最该带走的七句话

1. 胜率 $\neq$ 期望等待。胜率是吸收概率（齐次方程，无 +1）；等待是吸收时间（非齐次，有 +1）。两者脱钩。
2. 状态必须是进度局面 (i, j) ，不能退化成完整串。这是全篇最硬的教训——用完整串当递归参数会丢马尔可夫记忆，跨边界提前终止就算错、角色轮换就串味。
3. ”首次出现的此刻进度”互斥（能求和）；”历史上出现过的位置”相容（会重复计数）。 Γ_m 对， (k, p) 错。
4. 鞅 = 吸收概率值函数 $F(s)$ （即 h_s ），不是赌资 W 、不是筛选 R_X 。 $F(S_{t \wedge \tau})$ 严格鞅；odds 是两个鞅的初值之比（不是过程之比）。
5. 等长用 Conway 差值公式（漂亮），不等长用矩阵闭式（通用）。别拿等长的尺子量不等长。
6. ground truth（联合链 + 代码）是最终裁判，单点验证、记忆中的”经典公式”、漂亮的自创递归都可能骗你。
7. 自重叠是非传递性的发动机——它让”没有最强模式”的循环成为可能，并把第一、第二部分串成一条线。

8.4 留待第三部分的方向

1. **从矩阵闭式坍缩到等长 Conway 公式**：显式证明 $|X| = |Y| = n$ 时 $e_0(I - \frac{M}{2})^{-1}R_X$ 化简为 $C(Y, Y) - C(Y, X)$ 。
2. **用鞅直接构造 Conway**：从 $F(S_t)$ 是鞅出发，找显式 harmonic 函数（等长时由重叠数构造）——这是”鞅证明 Penney”的标准风格。
3. **L 作为互相关 / 卷积的物理意义**（本篇未展开）： $C(X, X)$ 是自相关（模式自己撞自己的回声）， $C(X, Y)$ 是互相关；学名 correlation polynomial (Guibas–Odlyzko)，和信号处理里的相关函数、检索里的相似度打分是同一个数学对象。这条线索值得单独一章。

致谢与归属。本篇推导主线、所有弯路、以及那个救场的关键警觉（坚持不等长、不信单点验证）由 Bo 驱动；Claude 负责对抗式判错、产出讲义，并贡献了一处教科书级的反面教材（凭记忆给错 L 定义、还声称”很明显”）；GPT 负责反例证伪与审稿修正（矩阵闭式、鞅的精确定位）；Python 联合链提供 ground truth 裁判。这正是多 AI 交叉审查方法论的一次真实实战——也是这份讲义除了数学之外，最想传递给你的东西。